

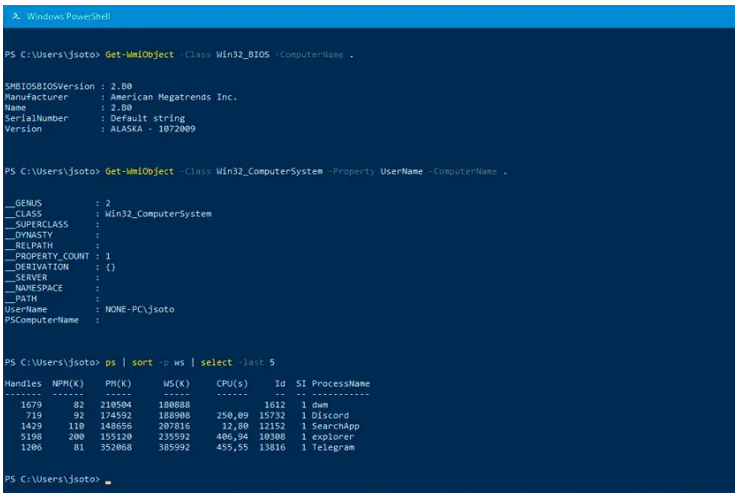
Administración de Sistemas Operativos

UD – 1

Ejercicios PowerShell

Ejercicios





Ejercicio 1: Monitor de servicios críticos

Conceptos: Get-Service, Where-Object, foreach, if, Write-Host, variables.

Crea un script llamado monitor_servicios.ps1 que realice lo siguiente:

- Defina en un array los nombres de los siguientes servicios críticos: "Spooler", "DHCP", "DNS" y "W32Time".
- Recorra el array con un bucle foreach y, para cada servicio, consulte su estado con Get-Service.
- Si el servicio está en ejecución (Status -eq "Running"), muestre un mensaje en **verde** indicando que está activo.
- Si el servicio está detenido, muestre un mensaje en **rojo** indicando la alerta.
- Lleve un contador de servicios detenidos y, al finalizar el bucle, muestre por pantalla el total de servicios caídos.

Requisitos:

- El script debe guardarse con extensión .ps1.
- Debe poder ejecutarse desde consola con .\monitor_servicios.ps1.
- Si aparece un error de política de ejecución, se debe aplicar Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser.

Ejercicio 2: Top 5 procesos por consumo de RAM

Conceptos: tuberías, `Sort-Object`, `Select-Object`, `Where-Object`, `Measure-Object`, propiedades calculadas.

Crea un script llamado `top_procesos.ps1` que realice lo siguiente:

- Liste los **5 procesos que más memoria RAM consumen** en el sistema.
- Muestre únicamente las columnas: `Name`, `Id` y una columna calculada llamada `RAM(MB)` que convierta el valor `WorkingSet64` a megabytes, redondeado a 2 decimales.
- A continuación, cuente cuántos procesos hay en total en el sistema utilizando `Measure-Object` y muestre el resultado.
- Pida al usuario, mediante `Read-Host`, el nombre de un proceso y compruebe con `Where-Object` y el operador `-like` si dicho proceso está en ejecución.
- Si existe, muestre un mensaje confirmándolo; si no, indique que no se encuentra.

Requisitos:

- Usar tuberías en lugar de bucles siempre que sea posible.
- La columna calculada debe usar `[math]::Round(...)`.

Ejercicio 3: Gestor de usuarios locales con menú interactivo

Conceptos: switch, do-while, Read-Host, Get-LocalUser, New-LocalUser, Remove-LocalUser, param().

Crea un script llamado gestion_usuarios_local.ps1 que muestre un menú repetitivo hasta que el usuario decida salir. El menú debe ser el siguiente:

```
text
```

1. Listar usuarios locales
2. Crear usuario
3. Eliminar usuario
4. Salir

Requisitos:

- El menú debe repetirse usando un bucle do-while hasta que se elija la opción 4.
- La selección de opción debe resolverse con una estructura switch.
- La opción **1** debe listar todos los usuarios locales mostrando al menos Name y Enabled.
- La opción **2** debe pedir el nombre del usuario con Read-Host y la contraseña con Read-Host -AsSecureString, y crear el usuario con New-LocalUser.
- La opción **3** debe pedir el nombre del usuario y eliminarlo con Remove-LocalUser.
- La opción **4** debe mostrar un mensaje de despedida y salir del bucle.
- Cualquier opción no válida debe mostrar un mensaje de error en rojo y volver a mostrar el menú.

Ejercicio 4: Inventario de logs con parámetros y exportación a CSV

Conceptos: param(), validación, redireccionamiento, Out-File, Export-Csv, Get-ChildItem, foreach, Test-Path.

Crea un script llamado inventario.ps1 que reciba dos parámetros desde la línea de comandos:

- $RutaLogs: ruta a la carpeta donde se encuentran los archivos .log (valor por defecto: C:\Logs). Debe validarse con [ValidateNotNullOrEmpty()].
- $Salida: ruta del archivo CSV de salida (valor por defecto: C:\temp\inventario.csv).

Requisitos:

- Al inicio, comprobar con Test-Path si la ruta $RutaLogs existe. Si no existe, mostrar un mensaje de error y salir con exit 1.
- Recorrer con un bucle foreach todos los archivos .log de esa carpeta.
- Por cada archivo, mostrar por pantalla su nombre y su tamaño en KB.
- Construir un objeto personalizado ([PSCustomObject]) por cada archivo con los campos: Nombre, TamañoKB y FechaModificacion.
- Exportar todos los objetos a un CSV en la ruta $Salida usando Export-Csv -NoTypeInformation.
- Redirigir los posibles errores del script a un archivo errores.txt mediante el operador 2>.

Requisitos adicionales:

- El script debe incluir una cabecera de comentarios con .SYNOPSIS, .DESCRIPTION, .PARAMETER y .EXAMPLE (según la plantilla del apartado 8 de los apuntes).
- Debe ejecutarse, por ejemplo, así:

```
.\inventario.ps1 -RutaLogs "C:\Logs" -Salida "C:\temp\inventario.csv"
```

Ejercicio 5: Creación masiva de usuarios en Active Directory desde CSV

Conceptos: integración de todos los anteriores — param(), Import-Csv, foreach, try/catch, New-ADUser, ConvertTo-SecureString, cabecera profesional, plantilla del manual.

Crea un script llamado crear_usuarios_ad.ps1 que cree usuarios en Active Directory a partir de un archivo CSV.

Requisitos:

1.El script debe seguir la **plantilla de script profesional** del apartado 8 de los apuntes e incluir al menos:

    - Cabecera con .SYNOPSIS, .DESCRIPTION, .PARAMETER, .EXAMPLE y .NOTES (autor, fecha, versión de PowerShell).
    - Zona de funciones (por ejemplo, Mostrar-Uso).
    - Zona de validación con param().
    - Sección principal.
- Recibir un único parámetro obligatorio: [Parameter(Mandatory=$true)][string] $RutaCSV.
- Validar con Test-Path que el archivo CSV existe. Si no existe, mostrar el uso del script y salir con exit 1.
- Importar el módulo ActiveDirectory con Import-Module.
- Leer el CSV con Import-Csv (formato: Nombre, SamAccountName, OU).
- Recorrer cada fila con foreach y crear el usuario con New-ADUser, asignando:
    - -Name, -SamAccountName, -UserPrincipalName (con formato usuario@contoso.com).
    - -Path con la OU indicada en el CSV.
    - -AccountPassword con ConvertTo-SecureString "P@sswOrd" -AsPlainText -Force.
    - -Enabled $true.

Archivo CSV de ejemplo (usuarios.csv**):**

CSV

```
Nombre,SamAccountName,OU
Ana Garcia,agarcia,"OU=Usuarios,DC=contoso,DC=com"
Luis Perez,lperez,"OU=Usuarios,DC=contoso,DC=com"
```

```
Maria Lopez,mlopez,"OU=Usuarios,DC=contoso,DC=com"
```

Ejemplo de ejecución:

```
.\crear_usuarios_ad.ps1 -RutaCSV "C:\temp\usuarios.csv"
```